

Stability-Aware Load Balancing with Autonomous Leadership Recovery for Resilient Fog-Assisted Industrial IoT Systems

Rangsimann Sattayarom, Ratchanon Wongwitutai, Phinnawat Yaemsanguan, Somchart Fugkeaw
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622771747@g.siit.tu.ac.th, 6622780268@g.siit.tu.ac.th, 6622770764@g.siit.tu.ac.th

Abstract—Fog computing enables low-latency processing for Industrial Internet of Things (IIoT) applications by extending computational resources closer to end devices. However, existing fog schedulers often suffer from coordinator failures, unstable workload allocation, and limited support for deadline-sensitive workloads under overload conditions. This paper proposes a stability-aware load-balancing framework with autonomous leadership recovery for resilient fog-assisted IIoT systems. The framework employs a hierarchical Master Fog Node (MFN)–Secondary Master Fog Node (SMFN) architecture, a Failure Resilience Index (FRI) for recovery-aware scheduling, and a Recovery Score (RS) for rapid coordinator replacement. In addition, a recovery-preserving collaborative resource-assistance mechanism dynamically recruits helper nodes for overloaded workloads while maintaining recovery readiness. To secure system operation, the framework integrates PUF-based device authentication, CRYSTALS-Kyber key establishment, and CP-ABE-based access control. Experimental results demonstrate superior workload latency, recovery responsiveness, deadline satisfaction, and system resilience compared with representative resource-aware, network-aware, and fault-tolerant fog scheduling schemes. The proposed framework provides an efficient and resilient workload-orchestration solution for dynamic fog-assisted IIoT environments.

Index Terms—Industrial Internet of Things (IIoT), Fog Computing, Adaptive Load Balancing, Fault Tolerance, Leader Election

I. INTRODUCTION

The Industrial Internet of Things (IIoT) is transforming modern manufacturing, industrial automation, smart transportation, and cyber–physical production systems by interconnecting large numbers of sensors, controllers, and intelligent devices. These systems continuously generate latency-sensitive data that must be processed in near real time to support predictive maintenance, anomaly detection, process optimization, and autonomous control. Although cloud computing provides abundant computational resources, transmitting all data to remote datacenters often introduces excessive latency and network congestion. Consequently, fog computing has emerged as an effective paradigm that extends computation and storage resources closer to end devices, enabling low-latency processing for time-critical industrial applications [1], [2], [11].

Efficient workload orchestration remains a fundamental challenge in large-scale fog-assisted IIoT environments. Existing studies have proposed various resource-aware, network-

aware, and fault-tolerant scheduling mechanisms to improve workload distribution and resource utilization [3], [6], [8]–[10], [12], [13]. However, most practical deployments operate under highly dynamic conditions characterized by fluctuating workloads, heterogeneous resources, varying network conditions, and unexpected node failures.

The first challenge is the reliance on centralized scheduling coordinators. Many fog architectures employ a master controller or scheduling node responsible for collecting system information and allocating workloads. Although effective under normal operation, coordinator failures can immediately disrupt workload assignment and create a single point of failure. Existing fault-aware scheduling schemes generally focus on task execution reliability and provide limited support for rapid coordinator recovery [12], [20].

The second challenge is maintaining accurate and stable scheduling decisions in dynamic fog environments. Resource availability, queue occupancy, communication latency, and workload intensity continuously change over time. Scheduling based solely on instantaneous resource information can result in workload oscillation, resource imbalance, and degraded performance under rapidly changing operating conditions [3], [13].

The third challenge concerns deadline-sensitive workloads generated by industrial monitoring and control systems. Existing scheduling approaches typically allocate workloads once and provide limited support for dynamically acquiring additional computational resources after deployment. Consequently, critical workloads may miss their deadlines even when spare resources remain available elsewhere in the fog infrastructure [8], [17].

To address these challenges, this paper proposes a stability-aware load-balancing framework with autonomous leadership recovery for resilient fog-assisted IIoT systems. The framework introduces a hierarchical coordination architecture consisting of a Master Fog Node (MFN) and a Secondary Master Fog Node (SMFN), a Failure Resilience Index (FRI) for recovery-aware scheduling, and a Recovery Score (RS) mechanism for rapid coordinator replacement. Furthermore, a recovery-preserving collaborative resource-assistance mechanism dynamically recruits helper nodes for overloaded workloads while preserving recovery-critical resources. To secure system operation, the framework integrates PUF-based device

authentication, CRYSTALS-Kyber key establishment, and CP-ABE-based result protection.

The major contributions of this paper are summarized as follows:

- 1) **Stability-Aware and Recovery-Aware Load Balancing:** We design a hierarchical MFN-SMFN scheduling architecture that jointly considers resource availability, communication latency, workload conditions, runtime stability, and failure resilience. The proposed Failure Resilience Index (FRI) enables recovery-aware scheduling decisions that improve workload distribution while reducing coordinator instability.
- 2) **Autonomous Leadership Recovery:** We develop a decentralized recovery mechanism that continuously replicates scheduling-state information across selected fog nodes. Combined with the proposed Recovery Score (RS), the framework enables rapid coordinator replacement without reconstructing global scheduling information, thereby minimizing service disruption during coordinator failures.
- 3) **Recovery-Preserving Collaborative Resource Assistance:** We propose a collaborative assistance mechanism that dynamically recruits helper nodes for overloaded and deadline-sensitive workloads while preserving recovery-capable resources. The proposed approach improves deadline satisfaction without compromising future recovery readiness.
- 4) **Integrated Security and Performance Evaluation:** We integrate PUF-based device authentication, post-quantum key establishment, and CP-ABE-based access control into the proposed framework. Experimental results demonstrate superior workload latency, recovery responsiveness, deadline satisfaction, and operational resilience compared with representative resource-aware, network-aware, and fault-tolerant fog scheduling schemes.

II. RELATED WORK

A. Load Balancing and Task Scheduling in Fog Computing

Efficient workload distribution is a fundamental requirement for fog-assisted IIoT systems. Existing studies have proposed a variety of load-balancing and task-scheduling mechanisms to improve resource utilization, reduce latency, and optimize quality-of-service performance. Kashani and Mahdipour [2] presented a comprehensive study of load-balancing algorithms in fog environments and highlighted the importance of adaptive resource allocation under heterogeneous computing conditions. Aldossary *et al.* [1] surveyed recent fog-computing load-balancing techniques and identified scalability, dynamic resource management, and fault tolerance as major open challenges.

Several studies have focused on workload-aware and resource-aware scheduling. Ala'anzy *et al.* [3] proposed a dynamic load-balancing framework for healthcare-oriented fog environments, while Jasim and Al-Raweshidy [6] developed an SDN-assisted load-balancing mechanism for real-time

healthcare systems. Yu *et al.* [8] introduced a location-aware scheduling framework that jointly considers fog and cloud resources, whereas Vijarania *et al.* [9] investigated energy-efficient workload allocation in integrated IoT-fog-cloud infrastructures. Other representative approaches include multi-objective optimization-based scheduling [10], smart-factory-oriented energy-aware load balancing [11], task-migration-based scheduling [12], and distributed learning-based resource allocation [13].

Although these approaches improve workload distribution and resource utilization, they primarily focus on scheduling optimization under normal operating conditions. Most assume the continuous availability of a centralized scheduler or coordinator and provide limited support for rapid recovery when scheduling entities fail. Furthermore, adaptive resource assistance for deadline-sensitive workloads is rarely considered after workload deployment.

B. Fault-Tolerant Scheduling and Resilient Edge Computing

To improve service availability in distributed environments, recent studies have investigated fault-tolerant scheduling and resilient edge-computing architectures. Xu *et al.* [15] proposed a fault-tolerant stream-processing framework for serverless edge environments that accommodates uncertain data rates. Cao *et al.* [16] developed a preference-aware fault-tolerant function-embedding mechanism for energy-harvesting edge systems. Taghinezhad-Niar and Taheri [17] introduced a cost-efficient fault-tolerant scheduling framework for deadline-constrained IoT workflows across edge-cloud infrastructures.

Several studies have focused on improving reliability through redundancy, coding, and intelligent task management. EdgeHydra [18] employs erasure coding to provide fault-tolerant edge data distribution, while Long *et al.* [19] proposed reinforcement-learning-based fault-aware task offloading for mobile edge computing. Chen *et al.* [20] presented a hybrid fault-tolerance-aware load-balancing mechanism that jointly considers workload distribution and reliability. More recently, Lu *et al.* [21] proposed a gossip-enhanced routing and fault-recovery framework for distributed networks.

Despite these advances, most existing solutions focus on task-level fault tolerance, redundancy management, or fault-aware scheduling. Coordinator survivability and autonomous recovery of scheduling services remain largely unexplored. In many existing systems, coordinator failures still require centralized recovery procedures, leader re-election, or external management services, which may introduce additional recovery delays and service disruption.

III. OUR PROPOSED SYSTEM

This section presents the proposed stability-aware load-balancing framework with autonomous leadership recovery for resilient fog-assisted IIoT systems. The framework integrates secure data acquisition, adaptive workload orchestration, decentralized coordinator recovery, and protected result outsourcing to ensure low-latency processing, continuous service

availability, and secure data management in dynamic industrial environments.

A. System Model

Fig. 1 illustrates the overall architecture of the proposed framework.

1) *System Entities*: The proposed framework consists of the following entities:

- 1) **Attribute Authority (AA)**: A trusted authority responsible for system initialization, generation of cryptographic parameters, issuance of attribute-based secret keys, and management of access-control policies for authorized users.
- 2) **Authorized Users (AUs)**: Legitimate users who submit workload requests and retrieve processing results. Access to outsourced results is granted only when user attributes satisfy the associated access-control policies.
- 3) **IIoT Devices**: Resource-constrained sensors, actuators, and industrial controllers equipped with Physical Unclonable Functions (PUFs). Devices generate sensing data and perform lightweight authentication and authenticated encryption before transmission.
- 4) **CRP Database Server**: A trusted repository that maintains challenge–response pairs (CRPs) for registered IIoT devices and supports PUF-based device authentication.
- 5) **Edge Gateway**: The entry point of the fog infrastructure that authenticates incoming device traffic, filters invalid packets, performs integrity verification, and aggregates validated data into workload batches.
- 6) **Master Fog Node (MFN)**: The primary scheduling coordinator responsible for workload allocation, helper-node selection, and orchestration of adaptive load-balancing operations across the fog infrastructure.
- 7) **Secondary Master Fog Node (SMFN)**: A backup coordination node that continuously collects runtime telemetry and maintains the global fog-node score table. The SMFN periodically replicates scheduling-state information to selected fog nodes to facilitate autonomous leadership recovery.
- 8) **Fog Nodes (FNs)**: Distributed computing servers that execute assigned workloads, report runtime status, and participate in collaborative processing and recovery operations when required.
- 9) **Cloud Storage**: An honest-but-curious storage provider that stores encrypted workload outputs, metadata, and system logs. Since all outsourced data remain encrypted, the cloud cannot access plaintext information.

B. System Process

The proposed scheme operates through six phases. Table I presents list of notations used in all system phases.

Phase I: System Initialization: This phase establishes the security, communication, and scheduling infrastructure of the proposed framework.

TABLE I
MAIN NOTATIONS

Notation	Description
λ	Security parameter
Ω	Attribute universe
MPK, MSK	CP-ABE public and master secret keys
SK_u	Attribute secret key of user u
F_j	Fog node j
$\Psi(F_j)$	Runtime state of fog node F_j
Q_j	Queue occupancy of fog node F_j
C_j	Computational capability of fog node F_j
M_j	Available memory resources of fog node F_j
L_j	Communication latency of fog node F_j
U_j	Runtime trust score of fog node F_j
R_j	Readiness score of fog node F_j
FRI_j	Failure Resilience Index of fog node F_j
RS_j	Recovery Score of fog node F_j
RC_j	Recovery capability score of fog node F_j
MFN	Master Fog Node
$SMFN$	Secondary Master Fog Node
$\mathcal{T}$	Global fog-node telemetry table
$\mathcal{C}_{epoch}$	Replicated scheduling-state snapshot
B_k	Workload batch k
$\Phi(B_k)$	Scheduling-aware workload profile
ω_k	Workload intensity score
δ_k	Deadline urgency score
ρ_k	Recovery-priority score
ΔC_k	Required assistance resources
$\mathcal{H}_k$	Helper-node set for workload B_k
$HScore_i$	Helper-node selection score
AB_i	Assistance budget of helper node F_i
CT_k	Encrypted workload result
$\mathcal{O}_k$	Outsourced result package

Step 1: Security Initialization: The Attribute Authority (AA) initializes the CP-ABE system by generating the public parameters and master secret key

$$(MPK, MSK) \leftarrow \text{Setup}(1^\lambda, \Omega) \quad (1)$$

where Ω denotes the attribute universe. Each authorized user u receives an attribute-bound secret key

$$SK_u \leftarrow \text{KeyGen}(MSK, \text{Attr}_u) \quad (2)$$

For device authentication, the CRP server generates challenge–response pairs

$$R_i = f_{\text{PUF}}(C_i) \quad (3)$$

and stores

$$CRP_i = (ID_i, C_i, R_i) \quad (4)$$

for each registered IIoT device.

Step 2: Post-Quantum Communication Initialization: Each gateway and fog node generates a CRYSTALS-Kyber key pair

$$(pk_x, sk_x) \leftarrow \text{KeyGen}_{\text{Kyber}}(1^\lambda) \quad (5)$$

where $x \in \text{GW}, F_1, \dots, F_n$. During registration, each IIoT device establishes a shared root key

$$(CT_i^{\text{KEM}}, K_i^{\text{root}}) \leftarrow \text{Encaps}(pk_{\text{GW}}) \quad (6)$$

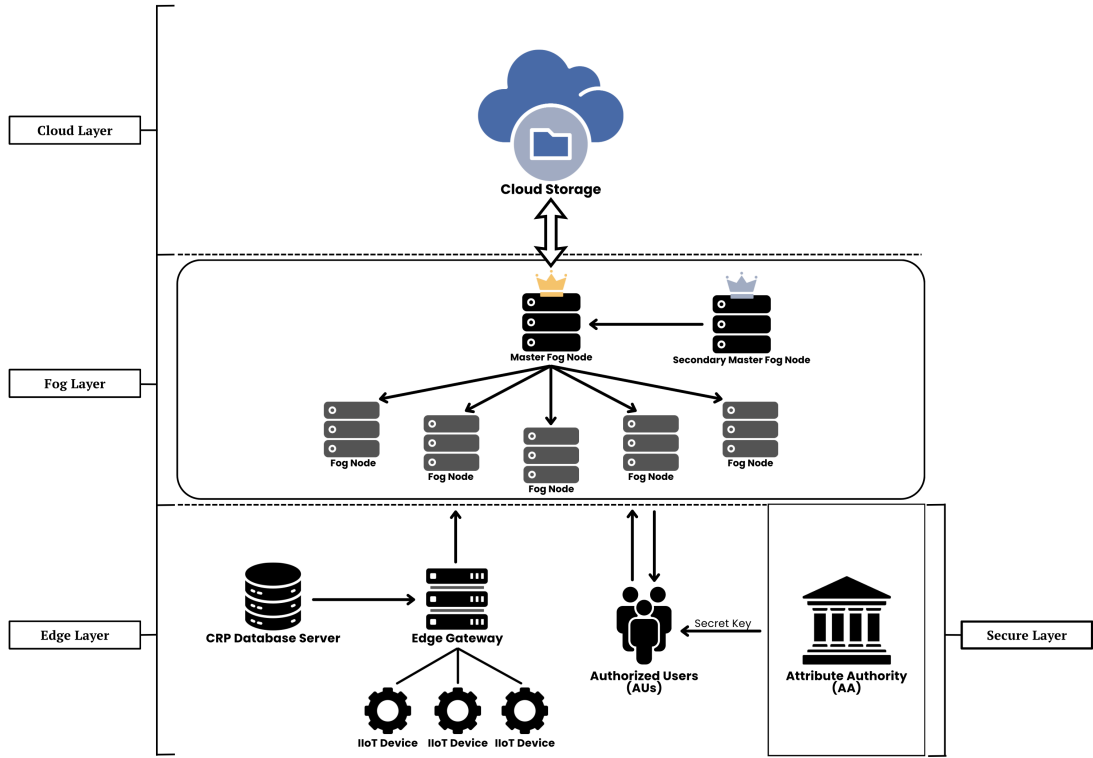


Fig. 1. System architecture of the proposed stability-aware load-balancing framework with autonomous leadership recovery for fog-assisted IIoT systems.

Step 3: Scheduling Infrastructure Initialization: Each fog node continuously monitors its runtime state

$$\Psi(F_j) = (Q_j, C_j, M_j, L_j, U_j) \quad (7)$$

where Q_j , C_j , M_j , L_j , and U_j denote queue occupancy, computational capability, available memory, communication latency, and trust score, respectively. Initially,

$$U_j^{(0)} = 1 \quad (8)$$

Phase II: Hardware-Bound IIoT Protection: This phase protects sensed data before it enters the fog infrastructure by combining PUF-based device identity with lightweight authenticated encryption.

Step 1: Hardware-Bound Key Derivation: Using the challenge-response pair established in Phase I, device D_i reconstructs its stable PUF secret

$$R_{secret,i} \leftarrow \text{FuzzyRep}(R_{noisy,i}, HD_i) \quad (9)$$

The device then derives a hardware-bound root key

$$K_{base,i} = \text{KDF}(K_{root,i} \parallel R_{secret,i} \parallel ID_i) \quad (10)$$

where $K_{root,i}$ denotes the post-quantum shared secret established during Phase I.

Step 2: Secure Packet Protection: For each sensed data item m_i , the device derives a fresh packet key

$$K_{p,i} = \text{KDF}(K_{base,i} \parallel t_i \parallel ctr_i) \quad (11)$$

where t_i and ctr_i denote the packet timestamp and counter.

The associated authenticated data is

$$AAD_i = ID_i \parallel t_i \parallel ctr_i \quad (12)$$

The device then encrypts and authenticates the packet using ChaCha20-Poly1305:

$$(CT_i, Tag_i) \leftarrow \text{AEAD.Enc}(K_{p,i}, N_i, m_i, AAD_i) \quad (13)$$

Step 3: Packet Generation: The protected packet is constructed as

$$P_i = ID_i, CT_i, N_i, Tag_i, AAD_i \quad (14)$$

and transmitted to the gateway for validation, workload formation, and scheduling in Phase III.

Phase III: Scheduling-Aware Workload Profiling and Batch Formation: This phase performs packet validation, replay filtering, and workload profiling before workload submission to the fog layer. By removing invalid traffic and constructing scheduling-aware workload descriptors at the gateway, the framework reduces unnecessary processing overhead and improves subsequent load-balancing decisions.

Step 1: Packet Validation: For each received packet

$$P_i = ID_i, CT_i, N_i, Tag_i, AAD_i \quad (15)$$

the gateway verifies the authenticated-encryption tag and freshness information contained in (t_i, ctr_i) . A packet is accepted only if integrity verification succeeds and no replay condition is detected. Invalid packets are discarded immediately.

Step 2: Micro-Batch Formation: All validated packets received within aggregation window t_{window} are grouped into

$$\mathcal{P}_{valid} = P_i \mid \text{Verify}(P_i) = 1 \quad (16)$$

To provide batch-level integrity, the gateway computes a commitment

$$GTag = \mathcal{H}(\text{Sort}(h_i) \parallel t_G) \quad (17)$$

where

$$h_i = \mathcal{H}(ID_i \parallel CT_i \parallel Tag_i) \quad (18)$$

and t_G denotes the gateway timestamp.

The resulting batch is

$$B_k = (BID, \mathcal{P}_{valid}, GTag, t_G) \quad (19)$$

Step 3: Scheduling-Aware Workload Profiling: To support adaptive scheduling and recovery operations, the gateway derives a workload profile

$$\Phi(B_k) = (S_k, \omega_k, \delta_k, \rho_k) \quad (20)$$

where S_k denotes the batch size, ω_k denotes workload intensity, δ_k denotes deadline urgency, and ρ_k denotes recovery priority.

The workload intensity is estimated as

$$\omega_k = \alpha_1 S_k + \alpha_2 V_k + \alpha_3 D_k \quad (21)$$

where V_k represents the aggregated payload volume and D_k denotes the expected processing complexity.

To prioritize critical workloads during overload or recovery conditions, the gateway computes

$$\rho_k = \beta_1 \delta_k + \beta_2 \omega_k + \beta_3 P_k \quad (22)$$

where P_k denotes the application-defined workload priority.

The resulting pair

$$(B_k, \Phi(B_k)) \quad (23)$$

is forwarded to the Master Fog Node (MFN). The workload profile is subsequently used for stability-aware scheduling, helper-node selection, and autonomous leadership recovery in the following phases.

Phase IV: Stability-Aware Load Balancing and Recovery-Oriented Coordination: This phase performs workload scheduling while maintaining recovery readiness throughout the fog infrastructure. Unlike conventional schedulers that optimize only resource utilization, the proposed framework jointly considers runtime stability, resource availability, and failure resilience.

Step 1: Stability-Aware Coordinator Election: Using the telemetry information collected in Phase I, each fog node F_j computes a coordination score

$$S_j^{coord} = \alpha_1 C_j + \alpha_2 M_j - \alpha_3 L_j + \alpha_4 U_j + \alpha_5 R_j \quad (24)$$

where C_j , M_j , L_j , U_j , and R_j denote computational capability, available memory, communication latency, trust score, and readiness factor, respectively.

The readiness factor is defined as

$$R_j = 1 - (\beta_1 \bar{Q}_j + \beta_2 \bar{M}_j + \beta_3 \bar{L}_j) \quad (25)$$

To reduce leadership oscillation, a stability penalty is applied:

$$\hat{S}_j^{coord} = S_j^{coord} - \gamma \Delta S_j \quad (26)$$

The fog nodes with the highest and second-highest scores are selected as the Master Fog Node (MFN) and Secondary Master Fog Node (SMFN), respectively.

Step 2: Failure Resilience Assessment: To quantify long-term reliability, each fog node computes a Failure Resilience Index (FRI):

$$FRI_j = \theta_1 U_j + \theta_2 R_j - \theta_3 FR_j \quad (27)$$

where FR_j denotes the normalized historical failure rate. A larger FRI_j indicates stronger operational stability and recovery capability.

Step 3: Recovery-Aware Workload Scheduling: For each workload pair $(B_k, \Phi(B_k))$, the MFN evaluates candidate fog nodes using

$$\begin{aligned} S_{j,k}^{sched} = & w_1 Q_j + w_2 L_j - w_3 C_j \\ & - w_4 M_j - w_5 U_j - w_6 FRI_j \\ & + w_7 \rho_k \end{aligned} \quad (28)$$

where ρ_k denotes the recovery-priority score of workload B_k .

To improve scheduling stability, exponential smoothing is applied:

$$\tilde{S}_{j,k} = \eta S_{j,k}^{sched} + (1 - \eta) \tilde{S}_{j,k}^{prev} \quad (29)$$

The workload is assigned to

$$F_k^* = \arg \min_{F_j} \tilde{S}_{j,k} \quad (30)$$

Step 4: Recovery-State Maintenance: To support autonomous leadership recovery, the SMFN periodically constructs and replicates a scheduling-state snapshot

$$\mathcal{C}_{epoch} = (Epoch, MFN, SMFN, \mathcal{T}) \quad (31)$$

where $\mathcal{T}$ denotes the latest global telemetry table.

Each cache node stores the most recent snapshot and computes a Recovery Score

$$RS_j = \mu_1 \hat{S}_j^{coord} + \mu_2 FRI_j + \mu_3 CF_j \quad (32)$$

where CF_j denotes cache freshness. The replicated recovery state enables immediate coordinator replacement without performing a global re-election process. The selected fog node subsequently executes workload batch B_k and reports runtime updates to the MFN and SMFN.

Phase V: Recovery-Preserving Collaborative Resource Assistance: This phase enables overloaded or deadline-sensitive workloads to acquire additional computational resources from neighboring fog nodes while preserving recovery readiness. Unlike conventional cooperative scheduling approaches that maximize resource utilization alone, the proposed framework jointly considers workload urgency and recovery capability.

Step 1: Assistance Request Generation: During execution, the hosting fog node continuously estimates the workload completion time. Assistance is triggered whenever

$$T_k^{est} > T_k^{deadline} \quad (33)$$

or when the workload exhibits high urgency score δ_k or recovery priority ρ_k .

The required computational deficit is estimated as

$$\Delta C_k = \max(0, C_k^{req} - C_k^{avail}) \quad (34)$$

and the assistance request

$$AR_k = (B_k, \Delta C_k, \delta_k, \rho_k) \quad (35)$$

is forwarded to the MFN.

Step 2: Recovery-Preserving Helper Selection: Using the latest telemetry table $\mathcal{T}$, the MFN evaluates candidate helper nodes through the recovery capability score

$$RC_i = \psi_1 FRI_i + \psi_2 RS_i \quad (36)$$

and computes

$$HScore_i = \lambda_1 A_i + \lambda_2 RC_i - \lambda_3 L_i \quad (37)$$

where A_i and L_i denote available computational capacity and communication latency, respectively.

To preserve autonomous recovery capability, nodes satisfying

$$RC_i \geq \tau_R \quad (38)$$

are preferentially reserved for future recovery operations whenever sufficient alternative helpers exist.

Step 3: Collaborative Assistance and Aggregation: For each selected helper node, an assistance budget is allocated as

$$AB_i = \kappa A_i FRI_i \quad (39)$$

The remaining workload is partitioned proportionally to the available assistance budgets:

$$W_i = \frac{AB_i}{\sum_{F_m \in \mathcal{H}_k} AB_m} \cdot \Delta C_k \quad (40)$$

Helper nodes process their assigned workload fragments in parallel and return the resulting outputs to the hosting fog node, which reconstructs the final result as

$$M_k = \text{Combine}(M_1, M_2, \dots, M_h) \quad (41)$$

By jointly optimizing deadline satisfaction and recovery readiness, the proposed mechanism improves workload responsiveness while preserving sufficient resources for future coordinator-recovery operations.

Phase VI: Autonomous Leadership Recovery and Secure Result Management: This phase maintains service continuity during coordinator failures and securely manages workload results after execution. By combining autonomous recovery with protected result outsourcing, the framework ensures uninterrupted operation and reliable result delivery.

Step 1: Coordinator Failure Detection: Each cache node continuously monitors the liveness of the MFN and SMFN through periodic heartbeat messages.

A coordinator is considered failed if

$$t_{now} - t_{last} > \tau_H \quad (42)$$

where t_{last} denotes the timestamp of the most recent heartbeat and τ_H is the failure-detection threshold.

Step 2: Autonomous Leadership Recovery: Upon detecting a coordinator failure, cache nodes utilize the replicated scheduling state

$$C_{epoch} = (Epoch, MFN, SMFN, \mathcal{T}) \quad (43)$$

generated in Phase IV.

Instead of performing a global re-election, each cache node computes the recovery score

$$RS_j = \mu_1 \hat{S}_j^{coord} + \mu_2 FRI_j + \mu_3 CF_j \quad (44)$$

where CF_j denotes cache freshness.

The replacement coordinator is selected as

$$MFN^{new} = \arg \max_{F_j} RS_j \quad (45)$$

The promoted node immediately resumes scheduling operations using the replicated state table, thereby minimizing service interruption.

Step 3: Result Aggregation: After workload execution or collaborative assistance, the hosting fog node collects all execution outputs

$$\mathcal{R}_k = R_1, R_2, \dots, R_h \quad (46)$$

and reconstructs the final workload result

$$R_k = \text{Combine}(\mathcal{R}_k) \quad (47)$$

A result digest is generated as

$$H_k = \mathcal{H}(R_k \parallel BID \parallel t_k) \quad (48)$$

Step 4: Secure Result Protection and Outsourcing: The result is encrypted using a randomly generated data key

$$K_k \leftarrow \{0, 1\}^\lambda \quad (49)$$

The data key is protected under the designated access policy

$$CT_k^{ABE} = \text{Encrypt}(MPK, K_k, \mathcal{P}_k) \quad (50)$$

The protected result package

$$\mathcal{O}_k = (CT_k, CT_k^{ABE}, H_k) \quad (51)$$

is outsourced to cloud storage.

Step 5: Authorized Result Retrieval: Upon receiving a retrieval request, an authorized user first recovers the data key

$$K_k = \text{Decrypt}(SK_u, CT_k^{ABE}) \quad (52)$$

and subsequently obtains

$$R_k = \text{Dec}(K_k, CT_k) \quad (53)$$

The user verifies integrity by recomputing

$$H'_k = \mathcal{H}(R_k \parallel BID \parallel t_k) \quad (54)$$

and accepting the result only if

$$H'_k = H_k \quad (55)$$

Thus, the framework simultaneously provides continuous operation under coordinator failures and secure delivery of workload results to authorized users.

C. Recovery Hierarchy

The proposed framework adopts a hierarchical recovery strategy to minimize service disruption under coordinator failures.

- **Level 1 Recovery:** If the MFN fails, the SMFN immediately assumes the coordinator role using the latest replicated scheduling state.
- **Level 2 Recovery:** If both the MFN and SMFN become unavailable, the cache node with the highest Recovery Score RS_j and the most recent scheduling-state snapshot is promoted as the new coordinator.
- **Level 3 Recovery:** If no eligible cache node satisfies the predefined recovery criteria, a decentralized voting procedure is invoked to elect a replacement coordinator.

Throughout the recovery process, worker fog nodes continue executing previously assigned workloads. Only new scheduling decisions are temporarily suspended until a new coordinator becomes available, thereby preserving service continuity and preventing workload interruption.

IV. SECURITY ANALYSIS

We consider an adversary capable of eavesdropping, modifying, replaying, and injecting messages over the communication network. Fog nodes and cloud servers may also experience failures or become unavailable. The Attribute Authority (AA), CRP database server, and underlying cryptographic primitives are assumed to be trusted.

Device Authentication and Clone Resistance: The framework employs PUF-based authentication to establish hardware-bound device identities. Communication keys are derived from device-specific PUF secrets and post-quantum shared secrets, preventing adversaries from cloning legitimate devices or generating valid authentication credentials without access to the original hardware.

Communication Security and Replay Resistance: Device-to-gateway communications are protected using CRYSTALS-Kyber key establishment and ChaCha20–Poly1305 authenticated encryption. Consequently, transmitted packets remain

confidential and any modification of ciphertexts, timestamps, counters, or authenticated data is detected during verification. Packet freshness is enforced through timestamp and counter validation, enabling replayed packets to be identified and discarded.

Access Control and Data Confidentiality: Outsourced processing results are protected through CP-ABE-based access control. The result-encryption key is encapsulated under an access policy defined by the AA, ensuring that only authorized users whose attributes satisfy the policy can recover the protected data. Therefore, unauthorized users and honest-but-curious cloud servers cannot access plaintext information.

Resilient Workload Orchestration: To eliminate single points of failure, the framework continuously replicates scheduling-state information across designated cache nodes. Combined with the Failure Resilience Index (FRI), Recovery Score (RS), and autonomous leadership recovery mechanism, coordinator replacement can be performed rapidly without reconstructing global scheduling information. Furthermore, recovery-preserving collaborative assistance reserves recovery-capable nodes during overload conditions, maintaining both workload responsiveness and future recovery readiness.

V. PERFORMANCE EVALUATION

This section evaluates the effectiveness of the proposed framework in terms of load-balancing efficiency, leadership recovery, deadline satisfaction, and overall system resilience.

A. Experimental Setup

The proposed framework was implemented in Python 3.12 and evaluated on a workstation with an Intel Core i7 processor, 32 GB RAM, and Ubuntu Linux. The testbed consists of one gateway, eight heterogeneous fog nodes, and one cloud server. Each fog node periodically reports telemetry information, including resource availability, queue occupancy, communication latency, trust score, and failure statistics. To emulate realistic industrial operating conditions, synthetic IIoT workloads are generated from 100–1000 virtual devices. Packet sizes range from 256 B to 8 KB, while workload deadlines vary between 100 ms and 5 s. The workload arrival rate is progressively increased to create congestion and overload scenarios. In addition, random MFN and SMFN failures are injected throughout the experiments to evaluate the effectiveness of the proposed autonomous recovery mechanism.

The proposed framework is compared with three representative fog-computing scheduling schemes: (i) the resource-aware load-balancing approach of Ala'anzy *et al.* [3], (ii) the SDN-assisted scheduling framework of Jasim and Al-Raweshidy [6], and (iii) the fault-tolerance-aware load-balancing scheme of Chen *et al.* [20]. All reported results are averaged over 20 independent runs.

B. Experiment 1: Workload Completion Latency

This experiment evaluates the effectiveness of the proposed stability-aware scheduling mechanism under increasing workload intensity. The workload arrival rate is varied from 10^2 to

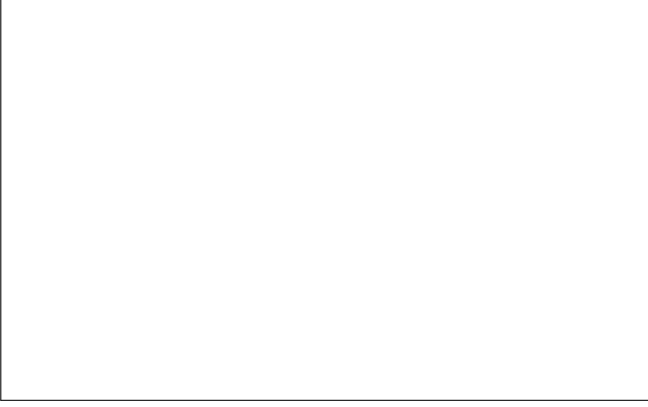


Fig. 2. Average workload completion latency versus workload arrival rate.

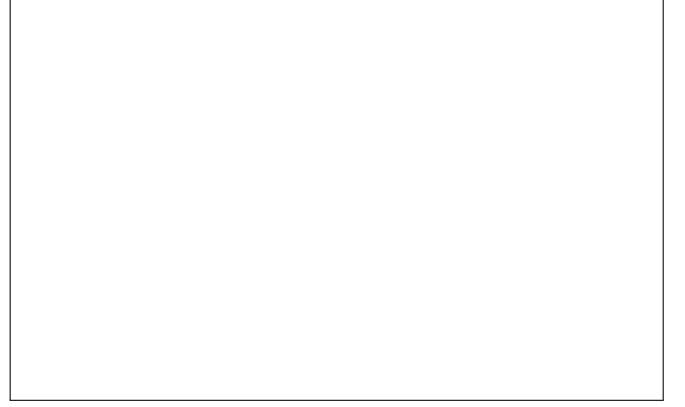


Fig. 3. Leadership recovery latency under coordinator failures.

10^4 workload batches, and the average workload completion latency is measured.

Fig. 2 compares the average workload completion latency of different schemes. Ala'anzy *et al.* [3] experience increasing latency under heavy workloads because scheduling decisions primarily rely on instantaneous resource availability. Jasim and Al-Raweshidy [6] achieve lower latency through network-aware workload distribution; however, scheduling performance degrades as workload dynamics become increasingly volatile. Chen *et al.* [20] improve workload distribution through fault-aware scheduling, but do not explicitly consider coordinator stability and recovery readiness.

The proposed framework consistently achieves the lowest workload completion latency. This improvement stems from the stability-aware scheduling strategy, which jointly considers resource availability, communication latency, workload conditions, trustworthiness, and failure resilience through the FRI metric. As workload intensity increases, the performance advantage becomes increasingly significant because unstable and overloaded nodes are proactively avoided during scheduling decisions.

C. Experiment 2: Leadership Recovery Latency

This experiment evaluates the responsiveness of the proposed autonomous leadership recovery mechanism. Random MFN and SMFN failures are injected during workload execution, and recovery latency is measured as the elapsed time between failure detection and restoration of scheduling functionality.

Fig. 3 shows that the proposed framework significantly outperforms all baseline schemes. Existing approaches rely on coordinator re-election, distributed coordination, or state reconstruction before scheduling services can resume. Consequently, recovery latency increases as the number of participating fog nodes grows.

In contrast, the proposed framework continuously maintains replicated scheduling-state snapshots and precomputed Recovery Scores (RS) across cache nodes. Upon detecting a coordinator failure, an eligible recovery node can immediately assume leadership using the latest scheduling state without

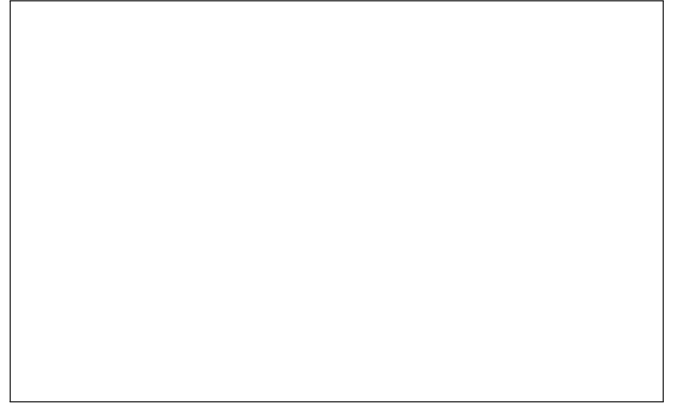


Fig. 4. Deadline-satisfaction ratio under overloaded operating conditions.

reconstructing global system information. As a result, the proposed framework achieves substantially lower recovery latency and minimizes service interruption.

D. Experiment 3: Deadline Satisfaction Under Overload

This experiment evaluates the effectiveness of the proposed recovery-preserving collaborative resource-assistance mechanism. The workload arrival rate is progressively increased until multiple fog nodes become saturated. The deadline-satisfaction ratio is measured as the percentage of workloads completed before their assigned deadlines.

As shown in Fig. 4, all schemes experience performance degradation as workload intensity increases. However, the proposed framework consistently maintains the highest deadline-satisfaction ratio. The baseline schemes primarily rely on workload reassignment or local resource management and therefore struggle when multiple fog nodes simultaneously approach saturation.

By contrast, the proposed framework dynamically recruits helper nodes based on both computational availability and recovery capability. Through the combined use of the Failure Resilience Index (FRI) and Recovery Score (RS), resource assistance is provided without exhausting recovery-critical nodes. Consequently, the framework improves workload re-



Fig. 5. Successful workload-completion ratio under combined overload and failure conditions.

sponsiveness while preserving sufficient recovery readiness for future failures.

E. Experiment 4: System Resilience Under Combined Failures and Overload

This experiment evaluates end-to-end system resilience under simultaneous workload surges and coordinator failures. The workload arrival rate is progressively increased while random coordinator failures are injected throughout the experiment. System resilience is measured using the successful workload-completion ratio.

Fig. 5 demonstrates that the proposed framework maintains the highest successful workload-completion ratio across all operating conditions. The baseline schemes experience noticeable performance degradation because coordinator recovery, workload balancing, and fault management are handled independently. Consequently, overload conditions and coordinator failures amplify each other's impact on system performance.

The proposed framework achieves superior resilience by integrating stability-aware scheduling, autonomous leadership recovery, and recovery-preserving collaborative assistance into a unified orchestration framework. Even under simultaneous overload and coordinator failures, workload execution continues with minimal disruption, resulting in significantly higher service availability and workload completion rates.

VI. CONCLUSION

This paper presented a stability-aware load-balancing framework with autonomous leadership recovery for resilient fog-assisted IIoT systems. The proposed framework integrates a hierarchical MFN-SMFN architecture, a Failure Resilience Index (FRI) for recovery-aware scheduling, a Recovery Score (RS) for rapid coordinator replacement, and a recovery-preserving collaborative assistance mechanism for deadline-sensitive workloads. In addition, PUF-based authentication, CRYSTALS-Kyber key establishment, and CP-ABE access control provide secure system operation. Experimental results demonstrated superior workload latency, recovery responsiveness, deadline satisfaction, and overall resilience compared

with representative fog-computing scheduling approaches. Future work will investigate predictive failure-aware scheduling and large-scale deployment in distributed industrial environments.

REFERENCES

- [1] D. Aldossary, E. Aldahasi, T. Balharith, and T. Helmy, "A Systematic Literature Review on Load-Balancing Techniques in Fog Computing: Architectures, Strategies, and Emerging Trends," *Computers*, vol. 14, no. 6, Art. no. 217, 2025, doi: 10.3390/computers14060217.
- [2] M. H. Kashani and E. Mahdipour, "Load Balancing Algorithms in Fog Computing," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1505–1521, 2023, doi: 10.1109/TSC.2022.3174475.
- [3] M. Ala'anzy, R. Zhanuzak, R. Akhmedov, N. Mohamed, and J. Al-Jaroodi, "Dynamic Load Balancing for Enhanced Network Performance in IoT-Enabled Smart Healthcare With Fog Computing," *IEEE Access*, vol. 12, pp. 188957–188975, 2024, doi: 10.1109/ACCESS.2024.3516362.
- [4] S. Fugkeaw, S. Rattagool, P. Jiangthiranan, and P. Pholwiset, "FPRESSO: Fast and Privacy-Preserving SSO Authentication With Dynamic Load Balancing for Multi-Cloud-Based Web Applications," *IEEE Access*, vol. 12, pp. 157888–157900, 2024, doi: 10.1109/ACCESS.2024.3485996.
- [5] S. Fugkeaw, R. Prasad Gupta, and K. Worapaluk, "Secure and Fine-Grained Access Control With Optimized Revocation for Outsourced IoT EHRs With Adaptive Load-Sharing in Fog-Assisted Cloud Environment," *IEEE Access*, vol. 12, pp. 82753–82768, 2024, doi: 10.1109/ACCESS.2024.3412754.
- [6] A. M. Jasim and H. Al-Raweshidy, "An Adaptive SDN-Based Load Balancing Method for Edge/Fog-Based Real-Time Healthcare Systems," *IEEE Systems Journal*, vol. 18, no. 2, pp. 1139–1150, June 2024, doi: 10.1109/JSYST.2024.3402156.
- [7] J. Singh, P. Singh, E. M. Amhoud, and M. Hedabou, "Energy-Efficient and Secure Load Balancing Technique for SDN-Enabled Fog Computing," *Sustainability*, vol. 14, no. 19, Art. no. 12951, 2022, doi: 10.3390/su141912951.
- [8] X. Yu, M. Zhu, M. Zhu, *et al.*, "A Novel Location-Aware Job Scheduling Framework for Optimizing Fog-Cloud IoT Systems: Insights From Dynamic Traffic Management," *Journal of Cloud Computing*, vol. 14, Art. no. 50, 2025, doi: 10.1186/s13677-025-00772-7.
- [9] M. Vijarania, S. Gupta, A. Agrawal, M. O. Adigun, S. A. Ajagbe, and J. B. Awotunde, "Energy Efficient Load-Balancing Mechanism in Integrated IoT-Fog-Cloud Environment," *Electronics*, vol. 12, no. 11, Art. no. 2543, 2023, doi: 10.3390/electronics12112543.
- [10] S. Ijaz, S. G. Ahmad, K. Ayyub, *et al.*, "Energy-Efficient Time and Cost Constraint Scheduling Algorithm Using Improved Multi-Objective Differential Evolution in Fog Computing," *The Journal of Supercomputing*, vol. 81, Art. no. 116, 2025, doi: 10.1007/s11227-024-06550-7.
- [11] J. Wan, B. Chen, S. Wang, M. Xia, D. Li, and C. Liu, "Fog Computing for Energy-Aware Load Balancing and Scheduling in Smart Factory," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 10, pp. 4548–4556, 2018, doi: 10.1109/TII.2018.2818932.
- [12] R. M. Singh, G. Sikka, and L. K. Awasthi, "LBATSM: Load Balancing Aware Task Selection and Migration Approach in Fog Computing Environment," *IEEE Systems Journal*, vol. 18, no. 2, pp. 796–804, June 2024, doi: 10.1109/JSYST.2024.3403673.
- [13] E. Oustad, "DIST: Distributed Learning-Based Energy-Efficient and Reliable Task Scheduling and Resource Allocation in Fog Computing," *IEEE Transactions on Services Computing*, vol. 18, no. 3, pp. 1336–1351, May–June 2025, doi: 10.1109/TSC.2025.3568255.
- [14] M. Maashi, "Elevating Survivability in Next-Gen IoT-Fog-Cloud Networks: Scheduling Optimization With the Metaheuristic Mountain Gazelle Algorithm," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 3802–3809, Feb. 2024, doi: 10.1109/TCE.2024.3371774.
- [15] Z. Xu *et al.*, "Efficient and Fault Tolerant Data Stream Processing With Uncertain Data Rates in Serverless Edge Computing," *IEEE Transactions on Services Computing*, vol. 19, no. 1, pp. 295–308, Jan.–Feb. 2026.
- [16] K. Cao, C. Tan, Y. Cui, and K. Li, "Preference-Aware Fault-Tolerant Function Embedding in Energy-Harvesting Serverless Edge Computing," *IEEE Transactions on Services Computing*, vol. 19, no. 2, pp. 1464–1477, Mar.–Apr. 2026.

- [17] A. Taghinezhad-Niar and J. Taheri, "Fault-Tolerant Cost-Efficient Scheduling for Energy and Deadline-Constrained IoT Workflows in Edge-Cloud Continuum," *IEEE Transactions on Services Computing*, vol. 18, no. 5, pp. 2892–2903, Sept.–Oct. 2025.
- [18] Q. He *et al.*, "EdgeHydra: Fault-Tolerant Edge Data Distribution Based on Erasure Coding," *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 1, pp. 29–42, Jan. 2025.
- [19] S. Long *et al.*, "Fault-Tolerant Aware Task Offloading Based on Reinforcement Learning in Mobile Edge Computing," *IEEE Transactions on Mobile Computing*, vol. 25, no. 5, pp. 6068–6082, May 2026.
- [20] S. Chen *et al.*, "A Hybrid Approach for Fault-Tolerance Aware Load Balancing in Fog Computing," *Cluster Computing*, vol. 27, pp. 1–20, 2024.
- [21] Y. Lu, Y. Zhang, Y. Zheng, R. Zhu, and W. Xiang, "Adaptive Gossip-Enhanced SIR Models for Real-Time Routing Optimization and Fault Tolerance in Distributed Networks," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 3819–3832, 2026, doi: 10.1109/TNSM.2026.3680838.
- [22] C. Peng, M. Luo, P. Vijayakumar, D. He, O. Said, and A. Tolba, "Multifunctional and Multidimensional Secure Data Aggregation Scheme in WSNs," *IEEE Internet of Things Journal*, vol. 9, no. 4, pp. 2657–2668, 2022.